

MODULE 4

Session 2 of 4

Dependency Injection & Service Lifetimes

The TMS Container Transient, Scoped, Singleton

Program.cs

```
builder.Services  
    .AddScoped<  
        IEnrollmentService,  
        EnrollmentService>();
```

```
builder.Services  
    .AddDbContext<  
        TMSDbContext>(...);
```

Session 2 The Service Container

By the end of this session, you will:

01

Explain dependency injection

Why ASP.NET Core is built around a container
not optional sugar

02

Choose lifetimes deliberately

Transient vs Scoped vs Singleton when each is
correct

03

Spot captive dependencies

Singleton capturing scoped work the silent
data-leak pattern

04

Keep DbContext scoped

Why a singleton DbContext corrupts tracking
under concurrent requests

05

Design TMS registrations

Enrollment services, DbContext, helps
consistent lifetime story

What Is Dependency Injection?

WITHOUT DI controller builds its own world

Inside action tight coupling

```
var db = new DatabaseConnection("Server=...");  
var svc = new EnrollmentService(db);
```

- Connection string in controller code
- Tests need a real database
- Every request may new() expensive collaborators

Ask the room

What does dependency mean here? Who creates EnrollmentService on each request, and when is it disposed?

WITH DI declare needs, container supplies

Constructor injection (C# 14)

```
[ApiController]  
[Route("api/enrollments")]  
  
public class EnrollmentsController(IEnrollmentService svc) :  
    ControllerBase { ... }  
  
// Program.cs  
  
builder.Services.AddScoped<  
    IEnrollmentService, EnrollmentService>();
```

Inversion of control: the controller never chooses how the service is constructed.

The Three Lifetimes

How long does one resolved instance live?

Transient

Every resolve → new instance

```
AddTransient<IRandomGenerator,  
    RandomGenerator>();
```

Same request, two resolves → two objects.

Scoped

One per HTTP request

```
AddScoped<IEnrollmentService,  
    EnrollmentService>();
```

Everything in one request shares one scope; disposed at the end.

Singleton

One for app lifetime

```
AddSingleton<IConfigReader,  
    ConfigReader>();
```

All requests share one instance until shutdown.

Decision table (cheat sheet)

Transient: every injection, disposed with scope. Scoped: once per request DbContext, request state. Singleton: once per app config, logging, caches (must be thread-safe).

Why DbContext Must Stay Scoped

Singleton DbContext looks fine in dev with one user it is a production incident under load.

Anti-pattern

```
// WRONG  
  
builder.Services.AddSingleton<TMSDbContext>();
```

EF Core registration

```
// CORRECT Scoped by default  
  
builder.Services.AddDbContext<  
    TMSDbContext>(o => o.UseNpgsql(  
        connectionString));
```

- Request A tracks Alice; Request B tracks Bob same DbContext → tracker chaos
- Connection pooling breaks when one long-lived context hoards the connection
- Concurrent SaveChanges: silent corruption or confusing failures

Rule: DbContext is always Scoped. No exceptions not about being careful; it is about concurrent requests.

If a learner pushes back

Singleton DbContext works until it does not at scale the change tracker is shared mutable state across requests. AddDbContext registers Scoped by default on purpose.

The Captive Dependency Bomb

Registrations

```
builder.Services.AddScoped<  
    IEnrollmentService, EnrollmentService>();  
  
builder.Services.AddSingleton<EnrollmentWorker>(); // BUG
```

Singleton worker is created once. First scoped `IEnrollmentService` gets injected and often stored in a field. Later requests reuse the worker stale scoped service.

TMS scenario two PUTs

Request 1: PUT /enrollments/1 body student=Alice

→ worker keeps scoped service in memory

Request 2: PUT /enrollments/2 body student=Bob

→ same worker → still sees Alice → accidental overwrite / leak

Whiteboard if the projector fails

Draw: Singleton worker box → arrow → Scoped enrollment service. Label the arrow: holds reference across requests. Full Mermaid lives in M4-ASP.NET-Core-Fundamentals(Captive Dependency).

The Captive Dependency Rule

A service may only depend on services with the same or longer lifetime.

Hierarchy (memory aid)

Singleton → depends on Singleton only

Scoped → Scoped or Singleton

Transient → Transient, Scoped, or Singleton

Singleton injecting Scoped

Development: host throws “Cannot consume scoped service from singleton.”

Production: validation is often off for performance bugs ship silently. Enable checks explicitly.

Catch at startup, not first request

```
builder.Host.UseDefaultServiceProvider(  
    o =>  
    {  
        o.ValidateScopes = true;  
        o.ValidateOnBuild = true;  
    });
```

Interview-sized takeaway

Scoped service that reads HttpContext user → Scoped, never Singleton otherwise every request shares one user identity.

Choosing the Right Lifetime TMS

Flow: mutable state? → per-request? → thread-safe for app-wide? (see spine flowchart)

TMSDbContext

Scoped

Connection + change tracker per request

IEnrollmentService

Scoped

Uses DbContext → must match Scoped

IConfigReader

Singleton

Immutable config after startup

ILogger<T>

Singleton

Thread-safe factory; shared pipeline

IGradeCalculator

Transient

Stateless helper; no scoped deps

DI Lifetimes Real-World Mistakes

Same lesson as Session 1: silent under light load, expensive or corrupt under concurrency not a style preference.

Scenario 1 “Current user” as Singleton

Program.cs

```
builder.Services.AddSingleton<ICurrentUserService,  
    CurrentUserService>(); // reads HttpContext
```

Result: every request shares one “current user” field Alice’s TMS session can look like Bob’s after parallel traffic.

Scenario 2 Singleton worker captures Scoped service

Program.cs

```
builder.Services.AddScoped<  
    IEnrollmentService, EnrollmentService>();  
  
builder.Services.AddSingleton<EnrollmentWorker>();
```

Result: first resolved scoped service gets pinned in the worker stale enrollments and cross-request leaks (captive dependency).

Contrast with previous slide (TMS table)

Per-user and DbContext-shaped work stays Scoped; app-wide, thread-safe collaborators stay Singleton; validate at build where you can.

DI Lifetimes for TMS

Registrations, DbContext scope, captive dependency fixes

- 1 Open Exercise 2 in M4-Lab-session-2. follow the workbook, not this slide
- 2 Align service lifetimes with DbContext (Scoped) and hosted work (Singleton)
- 3 Prove captive dependency is gone: ValidateOnBuild + a failing test path if applicable
- 4 Checkpoint: lifetime story is defensible before Session 3

Session 2 Summary What You Learned

Dependency injection

Controllers declare needs; the container constructs and disposes collaborators inversion of control for TMS endpoints.

Three lifetimes

Transient (every resolve), Scoped (per HTTP request), Singleton (app lifetime, must be thread-safe).

DbContext rule

Always Scoped shared tracker across requests is corruption under load, not a discipline problem.

Captive dependency

Singleton must not capture Scoped stale state, leaks, wrong enrollments when traffic overlaps.

Validation switches

ValidateScopes + ValidateOnBuild catch bad graphs at startup, not only in Development defaults.

TMS registrations

Align EnrollmentService with DbContext; config/logging singleton; stateless helpers transient when safe.

Reflection Before You Close the Laptop

- Why can't you inject a scoped service into a singleton?
- How would you know your code has this bug in production?
- What monitoring might hint at captive dependency behaviour?

Pair practice after reflection: 30 seconds each one person explains lifetimes for a TMS enrollment flow, the other asks “what if this were a singleton?”

What This Looks Like in an Interview

The captive dependency is a famous production bug. If you can explain it clearly "A singleton holds a reference to a scoped service, preventing it from being disposed per request, causing state leakage under concurrent load" interviewers recognize someone who has built real systems. Practice saying that sentence out loud.

Add the user-context trap: "Anything that reads HttpContext for the current user must be Scoped Singleton would share one identity across every request."

Coming Up in Session 3

Configuration & the Options Pattern

How services get connection strings, feature flags, and retry settings. Options makes missing configuration fail at startup with a clear error not three hours after deploy when the first user hits the endpoint.

```
IOptions<T>
```

```
IOptionsSnapshot<T>
```

```
IConfiguration
```

```
Bind()
```

```
ValidateDataAnnotations()
```